

MAE 146: Intro to ML with Python

Homework #3

Prof. Michael Tolley

Assigned: April 25, 2026
Due: April 30, 2026

Instructions:

- Homework submission is through Gradescope. Please leave extra time for submission, you can always re-submit up until the deadline.
- Your solution should not just give an answer, you should demonstrate how you got the answer.
- **For analytical problems**, submit handwritten answers (no typed responses).
- **For programming problems**, append any Python code you have used to solve the problem to your homework, and show any inputs you gave to your code, and any outputs it generated. You could do this by writing your solution in a Jupyter notebook, or by copying the output from your IDE (like Spyder) which may be easier for debugging. Either way, please add a pdf of your combined solution to your Gradescope submission (there will not be a separate code submission for this assignment).
- If you ever think a problem is stated incorrectly, not enough information is given, or it is impossible to solve, don't panic! Just make a reasonable assumption that will allow you to solve the problem, clearly stating your assumption(s), or indicate why it is not possible to solve the problem.
- For any questions regarding the assignment, please attend office hours (listed in the syllabus), or post them to Piazza.
- For initial homework submissions, we will only check that you have made an honest effort for each problem (e.g., that you did not just repeat the problem statement).
- After submission, you will have the opportunity to correct your solutions, and submit your corrections with the next homework (e.g., attach your HW1 corrections to the end of your submission for HW2), using the posted solutions as a guide.
- The corrected previous week's assignment should show handwritten corrections on top of the original submission (as if you were giving feedback on your own homework), as well as corrected code/plots for coding problems (corrected code is not required if your initial submission was correct).
- You are encouraged to discuss the homework with classmates, however your written submissions should be your own work. Completing your own assignments is the best way to learn the material and prepare for exams!

Analytical Problems (24 Points)

1. **Linear Support Vector Machines.** (16 points) In this problem, we will investigate the minimization of a training objective for a (Hard-Margin) Support Vector Machine and its relationship with a (Soft-Margin) SVM. Here, we only consider the case without the offset parameter θ_0 (that is, we set it equal to 0), and, for simplicity, assume that our dataset reduces to just one point $S_1 = \{(\mathbf{x}, y)\}$.

- (a) (4 points) The formulation of a **Hard-Margin SVM** can be given as the solution to a *constrained optimization problem* of the form:

$$\min_{\boldsymbol{\theta}} \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2$$

$$y(\boldsymbol{\theta} \cdot \mathbf{x}) \geq 1,$$

for a given parameter $\lambda > 0$. By solving this problem, what do we achieve? Reason why a solution $\hat{\boldsymbol{\theta}}$ of this problem will necessarily satisfy that $y(\hat{\boldsymbol{\theta}} \cdot \mathbf{x}) = 1$. Does λ have any effect on the solution to this problem?

- (b) **Minimizing Loss, Case 1** (3 points) In class, we have seen that a **Soft-Margin SVM** translates into an optimization that balances the average hinge loss over the examples and a regularization term that tries to keep the parameters small (and hence maximizes the margins). This balance is set by the regularization parameter $\lambda > 0$, and, therefore, the goal of the optimization is to find $\hat{\boldsymbol{\theta}}$ that solves:

$$\min_{\boldsymbol{\theta}} J_{\lambda}(\boldsymbol{\theta}) = \text{Loss}_h(y(\boldsymbol{\theta} \cdot \mathbf{x})) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2$$

First, suppose that the optimizer (solution) to the problem, $\hat{\boldsymbol{\theta}}$, satisfies $\text{Loss}_h(y(\hat{\boldsymbol{\theta}} \cdot \mathbf{x})) > 0$. Under this hypothesis, find the analytical solution to the optimization problem, and express $\hat{\boldsymbol{\theta}}$ in terms of $\mathbf{x}, y$ and λ .

- (c) **Minimizing Loss, Numerical Example 1** (1 point) Consider minimizing the Soft-Margin objective function from part (b) for the following numerical example:

$$\lambda = 0.5, \quad y = 1, \quad \mathbf{x} = (1, 0)^{\top}.$$

Note that this is a classification problem where points (examples) lie in a two-dimensional space. Hence $\hat{\boldsymbol{\theta}}$ would be a two-dimensional vector. Let $\hat{\boldsymbol{\theta}} = (\hat{\theta}_1, \hat{\theta}_2)^{\top}$, where $\hat{\theta}_1, \hat{\theta}_2$ are the first and second components of $\hat{\boldsymbol{\theta}}$, respectively. Using your result from the previous part, solve for $\hat{\theta}_1, \hat{\theta}_2$ assuming that $\text{Loss}_h(y(\hat{\boldsymbol{\theta}} \cdot \mathbf{x})) > 0$ (i.e., assuming that the given point is misclassified).

- (d) **Minimizing Loss, Numerical Example 2** (4 points) Verify that the solution to part (c) is not consistent with the assumption $\text{Loss}_h(y(\hat{\boldsymbol{\theta}} \cdot \mathbf{x})) > 0$, for the given values of λ, y , and $\mathbf{x}$ (i.e., the optimized $\hat{\boldsymbol{\theta}}$ has no loss).

Similarly, the Hard-Margin SVM has no loss (by definition), so we cannot use the approach of Parts b) & c) to find $\hat{\boldsymbol{\theta}}$. Instead, the solution can be found by a process that aims to find the best solution on the “boundary” of the constraint set:

- i. First, assume that $y(\hat{\boldsymbol{\theta}} \cdot \mathbf{x}) = 1$ (this defines the boundary of the constraint set).
- ii. From this equation, solve for $\hat{\theta}_1$ as a function of $\hat{\theta}_2$, using the given $\mathbf{x}$ which we know must be on the boundary.
- iii. Now, replace $\hat{\theta}_1$ in the objective function to write it purely as a function of $\hat{\theta}_2$.
- iv. Solve for the optimizer of the new function of $\hat{\theta}_2$. By doing this, we have “reduced” the optimization problem in part (b) to an optimization only wrt $\hat{\theta}_2$ and without constraints.
- v. Finally, substitute in for $\hat{\theta}_1$.

Note: This is in fact how you solve the Hard SVM.

- (e) **Soft SVM and misclassification** (2 points) Now, let $\hat{\boldsymbol{\theta}} = \hat{\boldsymbol{\theta}}(\lambda)$ be the solution to the **Soft-Margin SVM** as a function of λ . For what value of $\|\mathbf{x}\|^2$ will the training example $(\mathbf{x}, y)$ be misclassified for $\hat{\boldsymbol{\theta}}(\lambda)$? *Note:* In this case, assume that you can follow the procedure of Part (b). Recall that the goal of a soft SVMs is not just the correct classification of a point but also to obtain a classifier such that its decision boundary is far from correctly classified points.
- (f) **Minimizing loss, Numerical Example 2** (2 points) Now, let $\hat{\boldsymbol{\theta}} = \hat{\boldsymbol{\theta}}(\lambda)$ be the solution to the soft SVM as a function of λ . Given a fixed x , what values of λ will incur a nonzero loss $\text{Loss}_h(y(\hat{\boldsymbol{\theta}}(\lambda) \cdot \mathbf{x})) > 0$? What is the maximum value of λ that will achieve zero loss? *Note:* This is the value of λ for which the solutions of the hard SVM and soft SVM coincide. A larger λ will lead to nonzero losses for the feature point.
2. **The passive-aggressive algorithm.** (Total points: 8) In this problem, we will try to understand the loss in the so-called Passive-Aggressive (PA) Perceptron algorithm. The passive-aggressive (PA) perceptron algorithm (without offset) responds to a labeled training example $(\mathbf{x}, y)$ finding $\boldsymbol{\theta}$ that directly minimizes

$$\frac{\lambda}{2} \|\boldsymbol{\theta} - \boldsymbol{\theta}^{(k)}\|^2 + \text{Loss}_h(y(\boldsymbol{\theta}^\top \mathbf{x})), \quad (1)$$

at every iteration, where $\boldsymbol{\theta}^{(k)}$ is the current value of the parameter $\boldsymbol{\theta}$ prior to encountering the training data point $(\mathbf{x}, y)$ and

$$\text{Loss}_h(y(\boldsymbol{\theta} \cdot \mathbf{x})) = \max\{0, 1 - y(\boldsymbol{\theta} \cdot \mathbf{x})\}$$

is the hinge loss. The form of the update is similar to the perceptron algorithm's, i.e., we start from $\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}_{start}$ and then

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \eta y \mathbf{x}, \quad \text{every time a new training point is processed.}$$

However, observe that the real-valued step-size parameter η in this update is no longer equal to one; it now depends on both $\boldsymbol{\theta}^{(k)}$ and the training example $(\mathbf{x}, y)$. Answer the following:

- (a) **Step-size in the PA perceptron algorithm** (3 points). Look at the PA update and assume that $\text{Loss}_h(y(\boldsymbol{\theta}^{(k)} \cdot \mathbf{x})) > 0$. What is exactly the value of η in terms of λ ? *Hint:* You can simplify the loss function in this case.
- (b) **Step-size in the PA perceptron algorithm** (2 points). Look at the PA update and assume that $\text{Loss}_h(y(\boldsymbol{\theta}^{(k)} \cdot \mathbf{x})) = 0$. What is now the value of η in terms of λ ?
- (c) **Stochastic gradient descent algorithm** (3 points). What is the difference between this PA algorithm and stochastic gradient descent applied to $\text{Loss}_h(y(\boldsymbol{\theta} \cdot \mathbf{x})) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2$? *Note:* For comparison, use a different symbol, ζ_k , to denote the learning rate of the SGA.

Programming Problems (24 points)

1. **(Hinge loss function, 4 points)** Start by implementing your hinge loss function in `project3.py`. This function takes in as parameters $\boldsymbol{\theta}$, θ_0 and a feature matrix in which the rows are feature vectors and the columns are individual features, and a vector of labels representing the actual sentiment of the corresponding feature vector. Recall that the hinge loss function associated with a classifier $h(\mathbf{x}^{(i)}; \boldsymbol{\theta}, \theta_0)$ such as $h(\mathbf{x}^{(i)}; \boldsymbol{\theta}, \theta_0) = \boldsymbol{\theta} \cdot \mathbf{x}^{(i)} + \theta_0$ is given as

$$\ell_{\text{hinge}}(y^{(i)}, \mathbf{x}^{(i)}) = \max\{0, 1 - y^{(i)} h(\mathbf{x}^{(i)}; \boldsymbol{\theta}, \theta_0)\}.$$

Note: The number of elements in each row of the feature matrix $(\mathbf{x}^{(i)})$ should be the same as the number of elements in $\boldsymbol{\theta}$. Then, do the following:

- (a) (Basic hinge loss function, 2 points) Implement the so-called basic hinge loss function `hinge_loss_single(feature_vector, label, theta, theta0)` for a single data-point. Suppose that you are given one row of your feature matrix representing the feature vector of a single data sample, and its label of $+1$ or -1 representing the ground truth sentiment of the data sample. *Note:* Implement first the function locally, then run the script `test.py` to validate its basic functionality.
- (b) (Complete hinge loss, 2 point) Implement the complete hinge loss for a full set of data `hinge_loss_full(feature_matrix, labels, theta, theta0)` Your input will be a full feature matrix this time, and you will have a vector of corresponding labels. The k^{th} row of the feature matrix corresponds to the k^{th} element of the labels vector. This function should return the appropriate loss of the classifier on the given dataset.
2. (The perceptron algorithm, 6 points) Do the following:
- (a) (Perceptron Single Step Update, 2 points) Start by implementing a single step update `perceptron_single_step_update(feature_vector, label, current_theta, current_theta0)` for the perceptron algorithm (implemented with the 0-1 loss seen in class). You will be given the feature vector as an array of numbers, the current θ and θ_0 parameters, and the correct label of the feature vector. The function should return a tuple in which the first element is the correctly updated value of θ and the second element is the correctly updated value of θ_0 .
Note: Because of numerical instabilities, it is preferable to identify 0 with a small range $[-\epsilon, \epsilon]$ (for example, $\epsilon = 10^{-7}$). That is, when x is a float, “ $x == 0$ ” should be checked with $|x| \leq \epsilon$.
- (b) (Full perceptron algorithm, 2 points) Now implement the full perceptron algorithm via a function `perceptron(feature_matrix, labels, T)`. You will be given the same feature matrix and labels array as you were given to calculate the complete hinge loss function. You will also be given T , the maximum number of times that you should iterate through the feature matrix before terminating the algorithm. Initialize θ and θ_0 to zero. This function should return a tuple in which the first element is the final value of θ and the second element is the value of θ_0 .
Hints: Directly call the function `perceptron_single_step_update`. Make sure you initialize θ to a 1D array of shape $(n,)$ and not a 2D array of shape $(1, n)$. You should call `get_order(feature_matrix.shape[0], n)`, and use the ordering to iterate the feature matrix in each iteration. (i.e., obtain the ordering first during each iteration of T and then iterate through the feature matrix). By following this approach and the same random seed (as included in the starter code) you will be able to compare your results with those of other students.
- (c) (Average perceptron algorithm, 2 points) The average perceptron will add a modification to the original perceptron algorithm: since the basic algorithm continues updating as the algorithm runs, nudging parameters in possibly conflicting directions, it is better to take an average of those parameters as the final answer. Every update of the algorithm is the same as before. The returned parameters θ , however, are an average of the θ s across the nT steps:

$$\theta_{\text{final}} = \frac{1}{nT}(\theta^{(1)} + \theta^{(2)} + \dots + \theta^{(nT)})$$

Implement the average perceptron algorithm. This function should be constructed similarly to the Full Perceptron Algorithm above, except that it should return the average values of θ and θ_0 .

Hints: Tracking a moving average through loops is difficult, but tracking a sum through loops is simple. You should call `get_order(feature_matrix.shape[0], n)`, and use the ordering to iterate through the feature matrix in each iteration. (i.e., obtain the ordering first during each iteration of T and then iterate the feature matrix, as discussed above). Feel free to re-use functions you have previously implemented in this assignment.

3. (The Pegasos algorithm, 4 points) Now you will implement an alternative learning algorithm, the

Pegasos algorithm. The following pseudo-code describes the Pegasos update rule.

```

Pegasos ( $\mathbf{x}^{(i)}, y^{(i)}, \lambda, \eta, \boldsymbol{\theta}$ ) :
    if  $y^{(i)}(\boldsymbol{\theta} \cdot \mathbf{x}^{(i)}) \leq 1$  then
        update  $\boldsymbol{\theta} \leftarrow (1 - \eta\lambda)\boldsymbol{\theta} + \eta y^{(i)} \mathbf{x}^{(i)}$ 
    else
        update  $\boldsymbol{\theta} \leftarrow (1 - \eta\lambda)\boldsymbol{\theta}$ 

```

The η parameter is a factor that decreases over time. The λ parameter is a regularizing parameter. When using a bias $\boldsymbol{\theta}_0$, the update policy becomes:

```

Pegasos ( $\mathbf{x}^{(i)}, y^{(i)}, \lambda, \eta, \boldsymbol{\theta}, \boldsymbol{\theta}_0$ ) :
    if  $y^{(i)}(\boldsymbol{\theta} \cdot \mathbf{x}^{(i)} + \boldsymbol{\theta}_0) \leq 1$  then
        update  $\boldsymbol{\theta} \leftarrow (1 - \eta\lambda)\boldsymbol{\theta} + \eta y^{(i)} \mathbf{x}^{(i)}$ 
        update  $\boldsymbol{\theta}_0 \leftarrow \boldsymbol{\theta}_0 + \eta y^{(i)}$ 
    else
        update  $\boldsymbol{\theta} \leftarrow (1 - \eta\lambda)\boldsymbol{\theta}$ 
        Do not update  $\boldsymbol{\theta}_0$ 

```

- (a) (Pegasos algorithm, single update, 2 points) First, implement a single step of the Pegasos algorithm. This function is very similar to the function that you implemented in Perceptron Single Step Update, except that it should utilize the Pegasos parameter update rules instead of those for perceptron. The function will also be passed a η and λ value to use for updates.
 - (b) (Full Pegasos algorithm, 2 points). Implement the full Pegasos algorithm. Assume you will be given the same feature matrix and labels array as you were given in Full Perceptron Algorithm. You will also be given T , the maximum number of times that you should iterate through the feature matrix before terminating the algorithm. Initialize $\boldsymbol{\theta}$ and $\boldsymbol{\theta}_0$ to zero. For each update, set $\eta = 1/\sqrt{t}$ where t is a counter for the number of updates performed so far (between 1 and nT inclusive). This function should return a tuple in which the first element is the final value of $\boldsymbol{\theta}$ and the second element
4. (**Algorithm Discussion**, 10 points) Once you have completed the implementation of the 3 learning algorithms, you should qualitatively verify your implementations.
- (a) (Plots, 8 points) In `main_HW3.py` we have included a block of code that you should uncomment. This code loads a 2D dataset from `toy_data.txt`, and trains your models using $T = 10$, $\lambda = 0.2$. `mainn_HW3.py` will compute $\boldsymbol{\theta}$ and $\boldsymbol{\theta}_0$ for each of the learning algorithms that you have written. Then, it will call `plot_toy_data` to plot the resulting model and boundary. Please provide the values of $\boldsymbol{\theta}$ and $\boldsymbol{\theta}_0$ for each of the 3 algorithms.
 - (b) (Convergence, 2 point) Since you have implemented three different learning algorithms for linear classification, it is interesting to investigate which algorithm will actually converge. Please run them with a larger number of iterations T to visually check whether each algorithm converges. You may also check whether the parameters in your theta converge to the first decimal place. Achieving higher precision convergence requires more iterations, but your conclusion should be the same. Which of the algorithms converge for this dataset?